

Q&A Assignment

1. I ran the tests locally and captured the behavior. The U-mode “S-privileged” probes are already implemented in `__ch2_bad_address.c`, `__ch2_bad_instruction.c`, and `__ch2_bad_register.c`.

What I observed:

- SBI: RustSBI 0.1.1 (QEMU platform 0.1.0). This is printed by the bootloader on startup.
- `__ch2_bad_address` (U-mode store to NULL):
 - o `scause = 0x7` (Store/AMO access fault), `stval = 0`
 - o the handler doesn't handle access faults, so it falls into `unknown_trap()` in `trap.c`, prints an error, and exits with -1
- `__ch2_bad_register` (U-mode `csrr sstatus`) and `__ch2_bad_instruction` (U-mode `sret`):
 - o Both are illegal instruction from U-mode
 - o The illegal-instruction exception is not delegated to S-mode (`medeleg` doesn't include it), so RustSBI handles it in M-mode and panics with “Illegal instruction” then shuts down
 - o Because of the panic, only one of these two shows before the system stops (I saw the panic at `mepc=0x80440002`, which corresponds to `ch2_bad_register`)

Where this behavior comes from

- `Trap.c` only handles:
 - o Page faults/misaligned for load/str
 - o `IllegalInstruction`

But since the run hits:

- o Store/amo access fault (in this case `scause 7`) which isn't handled, its an unknown trap
- o Illegal instructions are not delegated to s-mode by `rustsbi`, so `sbi` just panics

2. A)

`a0` is the pointer to the process's `trapfram`. This is passed by `usertrapret()` as the sole argument

`a1` is for the user page table value for `satp` (SV39 `satp` value)

At runtime `a1` is whatever was already in the register (probably 0).

- B) `sfence.vma` flushes the TLB (address translation cache) for all virtual addresses. It is required after writing `satp` so that stale translations from the old page table do not remain.
- C) The comment means “don’t restore user `a0` from the trapframe yet,” because `a0` is being used as the pointer to the trapframe.

`a0` holds the trapframe pointer (the trapframe virtual address). All the loads (eg. `Ld ra, 40(a0)`) use that pointer. `A0` original value was saved at `112(a0)`, then moved into `sscratch`.

After all other `csrrw a0, sscratch`, `a0` swaps the user’s `a0` back into `a0` and puts the trapframe pointer into `sscratch`

D) The privilege switch happens at `sret` (last instruction in `userret`). `usertrapret()` sets `sstatus.SPP = 0` (U-mode), so `sret` returns to user state and jumps to `sepc`

E) After `csrrw a0, sscratch`, `a0` in `uservec` → `a0` = trapframe pointer (TRAPFRAME address). `sscratch` = the user’s original `a0` value. It swaps them so the handler can use `a0` as the trapframe base while preserving the user’s `a0` for later restore.

F) The save block starts at offset 40 (the `ra` slot) because the trapframe layout reserves the first fields for kernel metadata:

- 0: `kernel_satp`
- 8: `kernel_sp`
- 16: `kernel_trap`
- 24: `epc`
- 32: `kernel_hartid`

Those are set by the kernel before returning to user mode (in `usertrapret()`), so `uservec` must not overwrite them. The first user register slot is `ra` at offset 40, so that’s where saving starts.

- Almost all user GPRs are saved, but not all in that block:
 - `a0` is not saved there because `a0` currently holds the trapframe pointer. The original `a0` was swapped into `sscratch` and is saved later at offset 112.
 - `sepc` is saved separately into offset 24 via `csrr t1, sepc; sd t1, 24(a0)`.

G) There is no explicit instruction in `trampoline.S` that “enters S-state.” The switch to S-mode happens automatically in hardware when a trap/interrupt/exception occurs in U-mode.

H) t0 becomes the kernel trap handler address (kernel_trap) stored in the trapframe at offset 16. That value was written earlier in usertrapret(). So after ld t0, 16(a0), t0 holds the function pointer to usertrap, and jr t0 jumps into the kernel's C trap handler.